AI loop engineering: from zero to enterprise — Intermediate

Portfolio projects · lessons 09–18

Each takes 1–3 days and ends with something to show an employer.

Project 1: Pocket-cart fix loop with a Stop-hook gate

Goal: Build a Ralph-style outer loop that fixes a 12-failure Python repo one failing test per iteration, stops only on pytest's exit code, and reverts every red change, the pattern behind coding agents that run unattended.

Uses lessons: 07, 08, 10, 11, 12

1. Generate pocket-cart with the course's seeded generator and confirm 18 tests with 12 failing before you change any file.

2. Write verify.sh that runs pytest -q and exits 0 only when all 18 tests pass; this script is the only stop signal the loop trusts.

3. Write ralph.py that takes the first failing test, starts a fresh model call with only that test's output and the spec, and commits when verify.sh exits 0.

4. Run git checkout -- . after any iteration where verify.sh goes red, and log the revert with the test name in progress.md.

5. Keep progress.md (one line per iteration) and feature_list.json (test id and status) so each fresh call knows what is already done.

6. Add a script Stop hook that runs verify.sh and rejects the end of the run while it fails, then cap the run at 30 iterations and a $2.00 max_budget_usd.

7. Run the loop three times from a clean checkout with the same seed; use the scripted model first (no key), then one live run only if .env has the keys.

Deliverable: Repo with ralph.py, verify.sh, the Stop hook and a README table for the 3 runs: iterations to green (12 fixes are needed, so explain every extra iteration and revert), reverts, tests passing after each iteration, and total_cost_usd for the live run.

Stretch: Add a no-progress detector that ends the run when the same test fails with the same diff for 3 iterations. · Generate a 60-test repo with another seed and report iterations per fixed test against the 18-test run.

Project 2: Refund desk that survives flaky APIs and retries

Goal: Make the support desk's refund write safe to repeat: retries with backoff, an idempotency key, and a fresh-context checker that stops out-of-policy refunds, measured under injected failures. It shows a write path that a hiring manager can trust.

Uses lessons: 04, 05, 13, 14

1. Start from the Basic desk loop and add issue_refund(order_id, amount, request_id) that writes to a refunds table in brightside_crm.sqlite (about 400 orders), with the arguments validated by pydantic.

2. Add a UNIQUE index on refunds.idempotency_key, where the key is the SHA-256 of order_id, amount and request_id, so a repeat call returns the stored row and writes nothing.

3. Wrap the write and the model call in a retry with 5 attempts, exponential backoff from 0.5 s capped at 8 s and full jitter; retry 429, 500, 503 and timeouts only, never a 4xx validation error.

4. Write a fault-injecting stub that fails 30% of writes with a 503 and 10% of model calls with a timeout, seeded so every run is identical.

5. Run 200 refund requests through the loop under the stub, sending 20 of them twice on purpose, and count the rows per request_id.

6. Add a checker: a second model call with fresh context that reads the returns policy (30-day window) and the order, and must answer approve or reject with a reason; after 2 maker retries a reject goes to a human queue.

7. Plant 50 requests, 10 of them outside the 30-day window, and count how many the checker stops before the write; the no-key run uses a scripted checker, and one recorded live run adds the real model's answers.

Deliverable: Repo with the desk loop, the refund tool, the fault stub and a README reporting: duplicate refunds across 200 requests (target 0), attempts per request as mean and p95, out-of-window refunds written (target 0), checker catch rate on the 10 planted cases, and total_cost_usd per approved refund from the live run.

Stretch: Add a circuit breaker that opens after 5 consecutive 503s and pauses the desk for 60 seconds, then show the breaker's effect on p95 latency. · Run the checker on a second model and report the disagreement rate across the 50 planted requests.

Project 3: Accounts-payable invoice matcher with a maker, a checker and resume

Goal: A finance-operations loop that matches supplier invoices to purchase orders, has a fresh checker confirm each posting, and survives a hard kill without posting twice. Its verifier is arithmetic to the cent, so the stop decision never depends on a model's opinion.

Uses lessons: 04, 06, 07, 12, 14

1. Generate a synthetic AP set with a seeded script: 60 purchase orders, 120 invoices, about 300 invoice lines and 15 planted mismatches (quantity off, unit price up 1 to 5%, duplicate invoice number, missing PO), with invented vendor names and contacts at name@example.com.

2. Write a deterministic verifier in plain Python: line total equals quantity times unit price to the cent, invoice total equals the sum of lines, and every line is within 0.5% of its PO price; it returns pass or the first failing rule.

3. Write the maker loop with tools lookup_po(po_number) and lookup_receipts(po_number) defined with pydantic schemas, giving each invoice at most 6 turns and a $0.20 cap, and stopping on verifier pass, escalation or cap.

4. Add a stuck detector that ends a round when the same invoice gets the same error twice, and a checker with fresh context that sees only the invoice, the PO and the proposal; a reject gives the maker one more round, then the invoice goes to an escalation queue.

5. Keep one row per invoice in an SQLite runs table (status, attempts, last_error, posted_at) and make the post step idempotent on the invoice number.

6. Kill the process with a hard kill after invoice 40, restart it, and check that the 40 finished invoices are not posted again.

7. Score the output against the planted list: caught mismatches, false alarms among the 105 clean invoices, and the escalation rate.

Deliverable: Repo with the matcher, verifier, checker, SQLite schema, the kill script and a README reporting: planted mismatches caught or escalated (target 15 of 15), false alarms out of 105 clean invoices, double postings after the kill (target 0), escalation rate, and the mean total_cost_usd per closed invoice.

Stretch: Add timeouts to lookup_po (20% injected) with the retry from the refund project, and show the per-invoice cap still holds. · Swap the scripted maker for one recorded live model run and publish the same table with the real model's numbers.

Project 4: Golden-set eval gate in CI for the desk loop

Goal: Put the desk loop behind a pull-request gate that replays recorded model calls, scores 30 golden tasks with pass^3, and fails the build when a planted regression lands. It is the gate a team needs before any prompt, tool or model change ships.

Uses lessons: 07, 14, 17, 18

1. Write 30 golden tasks as JSON, each with a deterministic end-state check (order status, refund row count or a pytest call), plus 10 loop-trap tasks (impossible goal, flaky tool, contradictory policy).

2. Record one live run of all 40 tasks with llm_replay so CI replays the recorded calls and needs no key.

3. Run each task 3 times and compute the success rate and pass^3 (a task counts only if all 3 runs pass) with one script that prints a single table.

4. Write gate.yaml with thresholds (success at least 0.90 and pass^3 at least 0.80 on the 30 tasks; each trap task stops within 8 turns) and make gate.py exit 1 when any threshold fails.

5. Add a CI workflow that runs gate.py on every pull request and uploads the results table as a build artifact.

6. Plant three regressions on separate branches: remove the idempotency check, take the verifier call off the stop path, and set max_turns to unlimited; run the gate on each branch.

7. Record the wall-clock seconds of the gate and the recorded token count per task, to show the gate is cheap enough to run on every pull request.

Deliverable: Repo with the tasks, cassettes, gate.py, gate.yaml and the CI file, plus a README table: success rate and pass^3 for the clean build, and for each planted regression the metric that moved and whether the gate blocked it (target 3 of 3), with gate runtime in seconds.

Stretch: Add a trajectory check that fails any task calling the same tool with the same arguments 3 times in a row. · Schedule a nightly live run and report the drift between its pass^3 and the replay gate's pass^3.